

Implementation and Performance Analysis of the ChaCha20 Stream Cipher

Student Name: Ajaya Kumar Patel

Roll Number: CRS2503

Course: Cryptographic and Security Implementations

July 29, 2026

Contents

1	Introduction	2
2	Implementation Details	2
3	RFC 8439 Validation	3
3.1	Test Parameters	3
3.2	Expected Ciphertext (RFC 8439)	3
3.3	Obtained Ciphertext	3
4	Benchmark Methodology	4
5	Benchmark Results	4
6	Performance Comparison	5
7	References	6

1 Introduction

This project implements the ChaCha20 stream cipher in C according to RFC 8439. The implementation supports encryption and decryption using a 256-bit key, a 96-bit nonce, and a 32-bit block counter. The implementation was verified using the official RFC 8439 test vector, and its performance was evaluated through execution-time and CPU-cycle measurements.

2 Implementation Details

Function	Purpose
<code>rotl32()</code>	Performs a 32-bit left rotation.
<code>pack4()</code>	Packs four bytes into one 32-bit little-endian word.
<code>unpack4()</code>	Converts one 32-bit word back into four bytes.
<code>quarter_round()</code>	Implements the ChaCha20 quarter-round operation.
<code>chacha20_block()</code>	Generates one 64-byte ChaCha20 keystream block.
<code>chacha20_encrypt()</code>	Encrypts or decrypts data by XORing the plaintext or ciphertext with the generated keystream.
<code>benchmark_chacha20()</code>	Measures the execution time and CPU cycles of the ChaCha20 implementation.

Table 1: Functions Used in the ChaCha20 Implementation

3 RFC 8439 Validation

3.1 Test Parameters

Parameter	Value
Key	00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
Nonce	00 00 00 00 00 00 00 00 4A 00 00 00 00
Initial Counter	1
Plaintext	"Ladies and Gentlemen of the class of '99: If I could offer you only one tip for the future, sunscreen would be it."

Table 2: RFC 8439 Test Vector Parameters

3.2 Expected Ciphertext (RFC 8439)

```
6E2E359A2568F98041BA0728DD0D6981
E97E7AEC1D4360C20A27AFCCFD9FAE0B
F91B65C5524733AB8F593DABCD62B357
1639D624E65152AB8F530C359F0861D8
07CA0DBF500D6A6156A38E088A22B65E
52BC514D16CCF806818CE91AB7793736
5AF90BBF74A35BE6B40B8EEDF2785E42
874D
```

3.3 Obtained Ciphertext

```
6E2E359A2568F98041BA0728DD0D6981
E97E7AEC1D4360C20A27AFCCFD9FAE0B
F91B65C5524733AB8F593DABCD62B357
1639D624E65152AB8F530C359F0861D8
07CA0DBF500D6A6156A38E088A22B65E
52BC514D16CCF806818CE91AB7793736
5AF90BBF74A35BE6B40B8EEDF2785E42
874D
```

The ciphertext generated by the implemented ChaCha20 algorithm exactly matches the official RFC 8439 test vector. This confirms that the implementation correctly follows the ChaCha20 specification and produces the expected output for the standard test parameters.

4 Benchmark Methodology

The performance of the ChaCha20 implementation was measured by encrypting the RFC 8439 test plaintext for **100,000 iterations**. Execution time was recorded using `clock_gettime()` with the `CLOCK_MONOTONIC` clock, while CPU cycles were measured using the `__rdtscp()` instruction.

To prevent compiler optimization from removing the benchmark loop, a checksum of the ciphertext was computed after the benchmark. The program was compiled using GCC with the `-O3` optimization level.

The average execution time and average CPU cycles were calculated as:

$$\text{Average Time (ns)} = \frac{\text{Total Execution Time (ns)}}{\text{Number of Iterations}} \quad (1)$$

$$\text{Average CPU Cycles} = \frac{\text{Total CPU Cycles}}{\text{Number of Iterations}} \quad (2)$$

The benchmark was performed on the system shown in Table 3.

Parameter	Specification
Processor	Intel(R) Core(TM) i5-10300H CPU @ 2.50GHz
Maximum CPU Frequency	4.50 GHz
Minimum CPU Frequency	800 MHz
Operating System	Ubuntu Linux
Compiler	GCC (-O3)
Iterations	100,000

Table 3: System Configuration Used for Benchmarking

5 Benchmark Results

The benchmark was executed using the methodology described in the previous section. The implementation was compiled with the `-O3` optimization level, and the average execution time and CPU cycles were measured over 100,000 iterations. The benchmark results are summarized in Table 4.

Metric	Value
Compiler Optimization	-O3
Iterations	100,000
Plaintext Size	114 bytes
Total Execution Time	24,662,849 ns
Average Execution Time	246.63 ns
Average CPU Cycles	615.56
Checksum	223

Table 4: Benchmark Results of the ChaCha20 Implementation

6 Performance Comparison

To evaluate the effect of compiler optimization, the benchmark was executed using both unoptimized (-O0) and optimized (-O3) builds. The results are presented in Table 5.

Optimization	Average Time (ns)	Average CPU Cycles
-O0	3584.42	8946.69
-O3	246.63	615.56

Table 5: Performance Comparison Between -O0 and -O3

The optimized implementation compiled with -O3 achieved significantly better performance than the unoptimized (-O0) version. The average execution time decreased from 3584.42 ns to 246.63 ns, while the average CPU cycles were reduced from 8946.69 to 615.56. This corresponds to an improvement of approximately $14.5\times$, demonstrating the effectiveness of compiler optimizations such as function inlining, loop optimization, and efficient register allocation.

7 References

1. Y. Nir and A. Langley, *ChaCha20 and Poly1305 for IETF Protocols*, RFC 8439, Internet Engineering Task Force (IETF), June 2018.
<https://www.rfc-editor.org/rfc/rfc8439>
2. OpenAI, *ChatGPT (GPT-5.5)*, used for understanding the ChaCha20 algorithm, implementation guidance, and code review.
<https://chatgpt.com/>
3. Ginurx, *chacha20-c: C Implementation of ChaCha20*, GitHub Repository.
<https://github.com/Ginurx/chacha20-c/blob/master/chacha20.c>